

Mục lục

1	Tổng quan hệ thống	7
1.1	Giới thiệu đề tài	7
1.1.1	Bối cảnh và vấn đề	7
1.1.2	Mục tiêu đề tài	7
1.1.3	Phạm vi ứng dụng	7
1.2	Công nghệ và thư viện sử dụng	8
1.2.1	Ngôn ngữ lập trình	8
1.2.2	Thư viện và framework	8
1.3	Kiến trúc tổng thể	8
1.3.1	Mô hình kiến trúc MVVM kết hợp Clean Architecture	8
1.3.2	Luồng đồng bộ dữ liệu (Unidirectional Data Flow)	9
1.3.3	Quản lý trạng thái với Resource Wrapper	10
1.3.4	Cấu trúc package	10
1.4	Mô hình dữ liệu tổng thể	10
1.4.1	Sơ đồ thực thể - quan hệ (ERD)	10
1.4.2	Giao tiếp với Server	11
1.5	Dependency Injection (Hilt)	11
1.6	Cấu hình và bảo mật	11
1.6.1	AndroidManifest và quyền truy cập	11
1.6.2	Cấu hình mạng (Network Security Config)	11
2	Xác thực và Quản lý hồ sơ người dùng	12
2.1	Màn hình khởi động (Splash Screen)	13
2.1.1	Luồng kiểm tra phiên đăng nhập	13
2.1.2	Chuyển hướng tự động (Auto-navigation)	13
2.2	Đăng ký tài khoản	13

2.2.1	Yêu cầu chức năng	13
2.2.2	Luồng xử lý và validation	13
2.2.3	Giao diện màn hình đăng ký	13
2.3	Đăng nhập	13
2.3.1	Đăng nhập bằng email và mật khẩu	13
2.3.2	Cơ chế JWT và lưu trữ Token	13
2.3.3	Tự động làm mới Token (Token Refresh)	13
2.4	Quên mật khẩu và OTP	13
2.4.1	Luồng gửi OTP qua email	13
2.4.2	Xác thực OTP và đặt lại mật khẩu	13
2.5	Xem và cập nhật hồ sơ cá nhân	13
2.5.1	Hiển thị thông tin tài khoản	13
2.5.2	Chỉnh sửa thông tin hồ sơ	13
2.5.3	Thay đổi ảnh đại diện	13
2.6	Cài đặt tài khoản	13
2.6.1	Đổi mật khẩu	13
2.6.2	Đăng xuất	13
3	Bài đăng và Tìm kiếm	14
3.1	Màn hình Trang chủ (Home Feed)	15
3.1.1	Hiển thị danh sách bài đăng	15
3.1.2	Phân trang và tải thêm (Pagination)	15
3.1.3	Gợi ý bài đăng (Recommendation)	15
3.2	Tìm kiếm bài đăng	15
3.2.1	Tìm kiếm theo từ khóa	15
3.2.2	Bộ lọc tìm kiếm (giá, diện tích, vị trí)	15
3.2.3	Tích hợp Elasticsearch	15
3.2.4	Lịch sử tương tác tìm kiếm	15
3.3	Chi tiết bài đăng	15
3.3.1	Hiển thị thông tin phòng trọ	15
3.3.2	Xem ảnh / video phòng trọ	15
3.3.3	Bản đồ vị trí (Google Maps)	15
3.3.4	Thông tin chủ trọ và liên hệ	15
3.3.5	Ghi nhật ký xem bài (Contact Log)	15

3.4	Đăng và quản lý bài đăng của tôi	15
3.4.1	Đăng bài mới (tải ảnh, video)	15
3.4.2	Chỉnh sửa bài đăng	15
3.4.3	Xóa bài đăng	15
3.4.4	Xem trạng thái bài đăng (PostStatus)	15
3.5	Bài đăng đã lưu	15
3.5.1	Lưu và bỏ lưu bài đăng	15
3.5.2	Danh sách bài đã lưu	15
3.6	Lịch sử xem	15
3.6.1	Ghi nhận lịch sử xem tự động	15
3.6.2	Hiển thị và xóa lịch sử	15
3.7	Đánh giá chủ trọ	15
3.7.1	Xem đánh giá và thống kê (RatingStats)	15
3.7.2	Gửi đánh giá mới	15
4	Chat và Gọi video	16
4.1	Kiến trúc hệ thống Chat	17
4.1.1	Tổng quan kiến trúc (Socket.IO + Room Database)	17
4.1.2	Mô hình dữ liệu cục bộ (ConversationEntity, MessageEntity)	17
4.1.3	Đồng bộ dữ liệu giữa Server và Local DB	17
4.2	Danh sách hội thoại	17
4.2.1	Hiển thị danh sách cuộc trò chuyện	17
4.2.2	Huy hiệu tin nhắn chưa đọc (Unread Badge)	17
4.2.3	Tìm kiếm và lọc hội thoại	17
4.3	Nhắn tin thời gian thực	17
4.3.1	Gửi và nhận tin nhắn văn bản	17
4.3.2	Gửi tệp đính kèm (ảnh, file)	17
4.3.3	Trạng thái tin nhắn (Sent / Delivered / Read)	17
4.3.4	Đánh dấu đã đọc (Mark as Read)	17
4.4	Thông báo đẩy (Push Notification)	17
4.4.1	Tích hợp Firebase Cloud Messaging (FCM)	17
4.4.2	Xử lý thông báo khi app ở foreground / background	17
4.4.3	Lưu token FCM lên Server	17
4.5	Gọi video (WebRTC)	17

4.5.1	Tổng quan công nghệ WebRTC và Coturn STUN/TURN	17
4.5.2	Luồng thiết lập cuộc gọi (Signaling qua Socket.IO)	17
4.5.3	Màn hình gọi đến (IncomingCallActivity)	17
4.5.4	Màn hình gọi video (VideoCallActivity)	17
4.5.5	Kết thúc và từ chối cuộc gọi	17
4.6	Dịch vụ nền cuộc gọi (CallForegroundService)	17
4.6.1	Foreground Service và Notification khi đang gọi	17
4.6.2	Xử lý cuộc gọi đến khi màn hình khóa	17
5	Quản trị hệ thống	18
5.1	Tổng quan chức năng quản trị	19
5.1.1	Vai trò và phân quyền trong hệ thống	19
5.1.2	Giao diện quản trị trên ứng dụng Android	19
5.2	Kiểm duyệt bài đăng	19
5.2.1	Luồng duyệt bài đăng mới	19
5.2.2	Từ chối và ẩn bài đăng (HidePostRequest)	19
5.2.3	Lịch sử kiểm duyệt (ModerationHistory)	19
5.2.4	Các trạng thái bài đăng (PostStatus)	19
5.3	Quản lý báo cáo vi phạm	19
5.3.1	Tiếp nhận báo cáo từ người dùng	19
5.3.2	Xử lý và phản hồi báo cáo	19
5.4	Quản lý gói đăng ký (Subscription)	19
5.4.1	Các loại gói dịch vụ	19
5.4.2	Xem và gia hạn gói đăng ký	19
5.4.3	Quyền lợi theo từng gói	19
5.5	Nhật ký liên hệ (Contact Log)	19
5.5.1	Ghi nhận lịch sử liên hệ chủ trọ	19
5.5.2	Thống kê và báo cáo hoạt động	19
5.6	Đánh giá tổng thể và hướng phát triển	19
5.6.1	Kết quả đạt được	19
5.6.2	Hạn chế và hướng phát triển	19

Danh sách hình vẽ

1.1	Sơ đồ kiến trúc Clean Architecture kết hợp MVVM của hệ thống	9
1.2	Luồng tương tác và luân chuyển dữ liệu trong hệ thống	9

Danh sách bảng

Chương 1

Tổng quan hệ thống

1.1 Giới thiệu đề tài

1.1.1 Bối cảnh và vấn đề

Hiện nay, nhu cầu tìm kiếm phòng trọ tại các thành phố lớn ngày càng tăng cao. Tuy nhiên, việc tìm kiếm thông tin phòng trọ thường gặp nhiều khó khăn như thông tin không rõ ràng, hình ảnh không chân thực, hoặc khó khăn trong việc liên lạc trực tiếp với chủ trọ. Các ứng dụng hiện tại thường thiếu các tính năng tương tác trực tiếp và quản lý tiện lợi.

1.1.2 Mục tiêu đề tài

Dự án "Trợ Tốt Android" được phát triển nhằm cung cấp một nền tảng trên thiết bị di động (hệ điều hành Android) giúp kết nối người thuê và chủ nhà trọ một cách nhanh chóng, minh bạch và an toàn. Các mục tiêu cụ thể bao gồm:

- Cung cấp thông tin phòng trọ chi tiết, có định vị bản đồ và đa phương tiện (ảnh, video).
- Tích hợp tính năng Chat thời gian thực (Real-time Chat) và Gọi Video (Video Call) giúp hai bên dễ dàng trao đổi và xem phòng từ xa.
- Cung cấp hệ thống quản lý tin đăng, đánh giá chủ trọ, và quản lý gói đăng ký (Subscription) cho tài khoản chủ trọ.

1.1.3 Phạm vi ứng dụng

Ứng dụng được thiết kế cho hệ điều hành Android, nhằm tới hai đối tượng người dùng chính:

- **Người thuê trọ:** Tìm kiếm, lọc phòng trọ, lưu tin, xem lịch sử, đánh giá, nhấn tin và gọi video với chủ trọ.
- **Chủ nhà trọ:** Đăng tin cho thuê, quản lý tin đăng, mua gói dịch vụ (subscription), nhận tin nhắn/cuộc gọi từ người thuê.

1.2 Công nghệ và thư viện sử dụng

1.2.1 Ngôn ngữ lập trình

Dự án được phát triển 100% bằng ngôn ngữ **Java** cho Android, đảm bảo tính ổn định và tương thích với các thiết bị Android từ API level 27 (Android 8.1) trở lên (Target SDK 36).

1.2.2 Thư viện và framework

Dự án tận dụng các thư viện và framework hiện đại, bao gồm:

- **UI Components:** AndroidX AppCompat, Material Design, ConstraintLayout, SwipeRefreshLayout, DotsIndicator (cho xem ảnh/video trượt), PhotoView (zoom ảnh).
- **Kiến trúc & Lifecycle:** ViewModel, LiveData, Navigation Component, Paging 3 (cho Paging & Infinite scroll).
- **Networking & API:** Retrofit 2, OkHttp 3 (kèm Logging Interceptor), Gson (chuyển đổi dữ liệu JSON).
- **Lập trình phản ứng (Reactive Programming):** RxJava 3 và RxAndroid để xử lý luồng dữ liệu bất đồng bộ.
- **Dependency Injection (DI):** Hilt (Dagger-Hilt) để quản lý dependency vòng đời ứng dụng.
- **Cơ sở dữ liệu cục bộ (Local DB):** Room Database (tích hợp RxJava 3) để lưu trữ tin nhắn, hội thoại (Chat) cho tính năng Offline/Caching.
- **Xử lý hình ảnh:** Glide (có tích hợp OkHttp3) cho việc tải và hiển thị hình ảnh mượt mà.
- **Bản đồ & Vị trí:** Google Play Services Maps và Location API.
- **Thời gian thực (Real-time):** Socket.IO Client (v2.1.2) cho nhắn tin, WebRTC (phiên bản 104.5112) kết hợp Coturn (STUN/TURN) cho Gọi Video.
- **Thông báo đẩy:** Firebase Cloud Messaging (FCM).
- **Bảo mật:** EncryptedSharedPreferences (Android Security Crypto) để lưu trữ JWT an toàn.

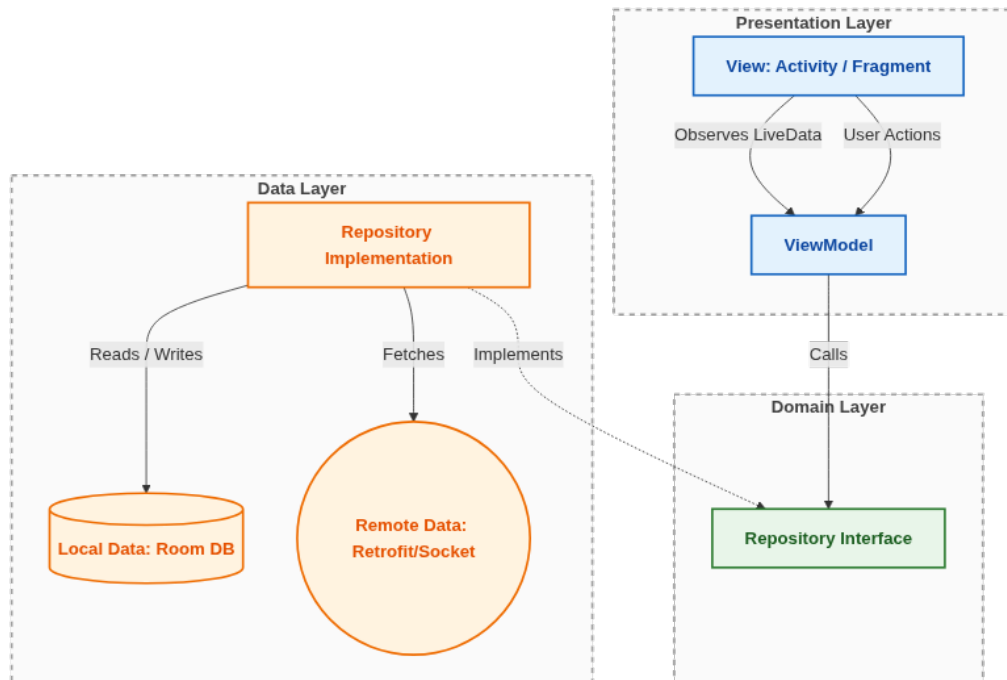
1.3 Kiến trúc tổng thể

1.3.1 Mô hình kiến trúc MVVM kết hợp Clean Architecture

Ứng dụng được thiết kế theo mô hình kiến trúc MVVM (Model - View - ViewModel) kết hợp với nguyên lý Clean Architecture, đảm bảo tính dễ bảo trì, dễ kiểm thử và khả năng mở rộng cao. Hệ thống được chia thành 3 lớp (layers) phân tách rõ ràng trách nhiệm:

- **Presentation Layer (Lớp Giao diện - ui):** Chứa các Activity và Fragment. Lớp này chỉ chịu trách nhiệm hiển thị dữ liệu và nhận tương tác. ViewBinding được dùng để ánh xạ giao diện an toàn. View quan sát (observe) LiveData từ ViewModel để tự động cập nhật UI, tuyệt đối không chứa logic nghiệp vụ hay gọi API trực tiếp.

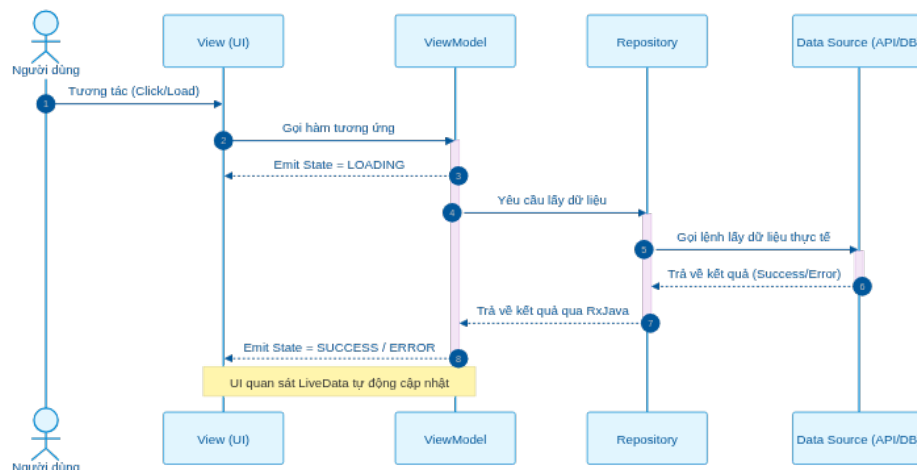
- **Domain Layer (Lớp Nghiệp vụ - repository):** Chứa các Repository interface đóng vai trò cầu nối. Trách nhiệm chính là trừu tượng hóa các thao tác xử lý logic, quyết định khi nào lấy dữ liệu từ Local và khi nào gọi lên Remote. Lớp này hoàn toàn độc lập và không phụ thuộc vào framework của Android.
- **Data Layer (Lớp Dữ liệu - data):** Nơi triển khai lấy dữ liệu thực tế. Bao gồm **Remote Data** (Retrofit API, Socket) và **Local Data** (Room DB, EncryptedPrefs).



Hình 1.1: Sơ đồ kiến trúc Clean Architecture kết hợp MVVM của hệ thống

1.3.2 Luồng đồng bộ dữ liệu (Unidirectional Data Flow)

Dự án áp dụng chặt chẽ cơ chế luồng dữ liệu một chiều (Unidirectional Data Flow) sử dụng RxJava kết hợp LiveData. Luồng lấy dữ liệu tuân thủ nguyên lý SSOT (Single Source of Truth):



Hình 1.2: Luồng tương tác và luân chuyển dữ liệu trong hệ thống

1. **Trigger:** View ghi nhận sự kiện người dùng (VD: nhấn nút Load) và gọi hàm tương ứng trong ViewModel.
2. **Process:** ViewModel phát tín hiệu Loading và gọi Repository để lấy dữ liệu.
3. **Fetch:** Repository thực thi Request tới Backend thông qua RxJava bằng luồng IO (`subscribeOn(Schedulers.io())`).
4. **Emit:** Dữ liệu sau khi xử lý được trả về dưới dạng `Single / Observable`, ViewModel đóng gói lại và đẩy vào `LiveData` trên Main Thread (`observeOn(AndroidSchedulers.mainThread())`).
5. **Update UI:** View (đang observe `LiveData`) tự động nhận được tín hiệu và render lại màn hình.

1.3.3 Quản lý trạng thái với Resource Wrapper

Để chuẩn hóa việc xử lý trạng thái UI trên toàn app, toàn bộ phản hồi từ Data Layer được bao bọc trong một Generic Class là `Resource<T>`. Lớp này quy định 3 trạng thái cốt lõi:

- **Status.LOADING:** Báo hiệu hệ thống đang xử lý. UI sẽ khóa các nút bấm không cần thiết và hiển thị `ProgressBar/Shimmer`.
- **Status.SUCCESS:** Chứa payload dữ liệu T chuẩn xác trả về từ server để tiến hành hiển thị lên View.
- **Status.ERROR:** Chứa chuỗi thông báo lỗi đã được chuẩn hóa bởi `ErrorHandler`. Hệ thống có khả năng tự động phân tích ngoại lệ (`SocketTimeout`, `HTTP 401`, `500...`) thành câu thông báo tường minh bằng tiếng Việt để hiển thị `Toast/Snackbar` cho người dùng.

1.3.4 Cấu trúc package

Source code được tổ chức theo tính năng (Feature-based Package) bên trong `com.troto.vn.trotoandroid`:

- **data:** Lớp dữ liệu gồm `local` (Room), `remote` (Retrofit/Socket), `model` (Entities, DTOs) và `repository`.
- **di:** Khai báo Dependency Injection container bằng Hilt (`AppModule`, `NetworkModule`).
- **ui:** Lớp giao diện, chia nhỏ theo màn hình: `auth`, `main` (chứa `home`, `search`, `chat`, `profile`), `postdetail`, `videocall...`
- **services:** Nơi chứa các Foreground/Background Services như FCM (`MyFirebaseMessagingService`) và cuộc gọi nền.
- **utils:** Các hằng số (`Constants`), quản lý phiên bản (`SessionManager`) và các hàm hỗ trợ độc lập.

1.4 Mô hình dữ liệu tổng thể

1.4.1 Sơ đồ thực thể - quan hệ (ERD)

Dữ liệu cục bộ chủ yếu phục vụ cho tính năng Chat (Offline/Caching). Trong Room Database, các Entity chính bao gồm: `ConversationEntity`, `MessageEntity`, `ConversationParticipantEntity`, và `MessageAttachmentEntity`.

1.4.2 Giao tiếp với Server

Giao tiếp chính sử dụng Retrofit trả về kiểu dữ liệu luồng của RxJava (`Single`, `Observable`). Base URL của hệ thống được đặt tại biến `Constants.BASE_URL`. Dữ liệu JSON trả về được parse tự động qua `GsonConverter`.

1.5 Dependency Injection (Hilt)

Dependency Injection (DI) trong ứng dụng được quản lý toàn diện bởi thư viện **Dagger-Hilt**. Lớp `App` được đánh dấu bằng `@HiltAndroidApp` để khởi tạo Hilt container. Hilt module được thiết lập trong package `di`:

- **AppModule**: Cung cấp các instance cấp độ ứng dụng như `SessionManager`, `AppDatabase` (Room), và các thể hiện (implementation) của `Repository`.
- **NetworkModule**: Cung cấp các network client như `OkHttpClient`, `Retrofit`, `Gson`, `ApiService`, `SocketIOManager`, kèm theo các `Interceptor` để xử lý xác thực (`AuthInterceptor`) và tự động nối lại token hết hạn (`TokenAuthenticator`).

1.6 Cấu hình và bảo mật

1.6.1 AndroidManifest và quyền truy cập

Ứng dụng khai báo chi tiết các quyền trong `AndroidManifest.xml` để đảm bảo tuân thủ bảo mật và trải nghiệm người dùng:

- **Mạng & Vị trí**: Quyền truy cập Internet, Mạng (`INTERNET`, `ACCESS_NETWORK_STATE`) và GPS (`ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`) cho tính năng tìm kiếm trên bản đồ.
- **Video Call & Chat**: Quyền truy cập Camera, Microphone (`CAMERA`, `RECORD_AUDIO`), tinh chỉnh âm thanh (`MODIFY_AUDIO_SETTINGS`).
- **Dịch vụ nền**: Quyền chạy Foreground Service (`FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_PHONE_CALL`) và quyền hiện UI toàn màn hình (`USE_FULL_SCREEN_INTENT`) phục vụ việc hiện luồng cuộc gọi đến khi khóa màn hình.
- **Thông báo**: Quyền hiển thị thông báo (`POST_NOTIFICATIONS`) cho Android 13+.

1.6.2 Cấu hình mạng (Network Security Config)

Ứng dụng được cấu hình Network Security (`network_security_config.xml`) cho phép traffic dạng ClearText (HTTP) ở môi trường phát triển (với domain `trototvn.lamdx4.servebeer.com`) đồng thời cho phép tin cậy các chứng chỉ bảo mật ở cấp độ hệ thống để giao tiếp an toàn qua HTTPS ở Production. Token người dùng (JWT) được lưu trữ an toàn mã hóa qua `EncryptedSharedPreferences`.

Chương 2

Xác thực và Quản lý hồ sơ người dùng

2.1 Màn hình khởi động (Splash Screen)

2.1.1 Luồng kiểm tra phiên đăng nhập

2.1.2 Chuyển hướng tự động (Auto-navigation)

2.2 Đăng ký tài khoản

2.2.1 Yêu cầu chức năng

2.2.2 Luồng xử lý và validation

2.2.3 Giao diện màn hình đăng ký

2.3 Đăng nhập

2.3.1 Đăng nhập bằng email và mật khẩu

2.3.2 Cơ chế JWT và lưu trữ Token

2.3.3 Tự động làm mới Token (Token Refresh)

2.4 Quên mật khẩu và OTP

2.4.1 Luồng gửi OTP qua email

2.4.2 Xác thực OTP và đặt lại mật khẩu

2.5 Xem và cập nhật hồ sơ cá nhân

2.5.1 Hiển thị thông tin tài khoản

Chương 3

Bài đăng và Tìm kiếm

3.1 Màn hình Trang chủ (Home Feed)

3.1.1 Hiển thị danh sách bài đăng

3.1.2 Phân trang và tải thêm (Pagination)

3.1.3 Gợi ý bài đăng (Recommendation)

3.2 Tìm kiếm bài đăng

3.2.1 Tìm kiếm theo từ khóa

3.2.2 Bộ lọc tìm kiếm (giá, diện tích, vị trí)

3.2.3 Tích hợp Elasticsearch

3.2.4 Lịch sử tương tác tìm kiếm

3.3 Chi tiết bài đăng

3.3.1 Hiển thị thông tin phòng trọ

3.3.2 Xem ảnh / video phòng trọ

3.3.3 Bản đồ vị trí (Google Maps)

3.3.4 Thông tin chủ trọ và liên hệ

3.3.5 Ghi nhật ký xem bài (Contact Log)

3.4 Đăng và quản lý bài đăng của tôi

3.4.1 Đăng bài mới (tải ảnh, video) ¹⁵

3.4.2 Chỉnh sửa bài đăng

Chương 4

Chat và Gọi video

4.1 Kiến trúc hệ thống Chat

4.1.1 Tổng quan kiến trúc (Socket.IO + Room Database)

4.1.2 Mô hình dữ liệu cục bộ (ConversationEntity, MessageEntity)

4.1.3 Đồng bộ dữ liệu giữa Server và Local DB

4.2 Danh sách hội thoại

4.2.1 Hiển thị danh sách cuộc trò chuyện

4.2.2 Huy hiệu tin nhắn chưa đọc (Unread Badge)

4.2.3 Tìm kiếm và lọc hội thoại

4.3 Nhắn tin thời gian thực

4.3.1 Gửi và nhận tin nhắn văn bản

4.3.2 Gửi tệp đính kèm (ảnh, file)

4.3.3 Trạng thái tin nhắn (Sent / Delivered / Read)

4.3.4 Đánh dấu đã đọc (Mark as Read)

4.4 Thông báo đẩy (Push Notification)

4.4.1 Tích hợp Firebase Cloud Messaging (FCM)

4.4.2 Xử lý thông báo khi app ở foreground / background

4.4.3 Lưu token FCM lên Server

4.5 Gọi video (WebRTC)

Chương 5

Quản trị hệ thống

5.1 Tổng quan chức năng quản trị

5.1.1 Vai trò và phân quyền trong hệ thống

5.1.2 Giao diện quản trị trên ứng dụng Android

5.2 Kiểm duyệt bài đăng

5.2.1 Luồng duyệt bài đăng mới

5.2.2 Từ chối và ẩn bài đăng (HidePostRequest)

5.2.3 Lịch sử kiểm duyệt (ModerationHistory)

5.2.4 Các trạng thái bài đăng (PostStatus)

5.3 Quản lý báo cáo vi phạm

5.3.1 Tiếp nhận báo cáo từ người dùng

5.3.2 Xử lý và phản hồi báo cáo

5.4 Quản lý gói đăng ký (Subscription)

5.4.1 Các loại gói dịch vụ

5.4.2 Xem và gia hạn gói đăng ký

5.4.3 Quyền lợi theo từng gói

5.5 Nhật ký liên hệ (Contact Log)

5.5.1 Ghi nhận lịch sử liên hệ chủ trọ

5.5.2 Thống kê và báo cáo hoạt động